R3.07
SQL dans un langage de programmation

Chapitre 5
Patrons DAO et DTO



Selena Lamari et Chouki Tibermacine

BUT 2 Informatique — IUT de Vannes — Université Bretagne Sud

Introduction

DAO (Data Access Object)

- Séparation de la logique métier et de l'accès aux données.
- Fournit une interface unifiée pour accéder à une source de données (BDD, fichiers, etc.).

DTO (Data Transfer Object)

- Objet simple utilisé pour transférer des données entre processus ou couches.
- Réduit le nombre d'appels réseau en regroupant les données.

Exemple DAO en Java (JDBC)

Interface DAO

```
public interface UserDAO {
    void addUser(User user);
    User getUser(int id);
    List<User> getAllUsers();
}
```

Exemple DAO en Java (Implémentation JDBC)

```java
public class UserDAOImpl implements UserDAO {
 private Connection connection;
 public UserDAOImpl(Connection connection) {
  this.connection = connection;
 }
 @Override
 public void addUser(User user) throws SQLException {
  String sql = "INSERT INTO users (name, email) VALUES (?, ?)";
  try (PreparedStatement stmt = connection.prepareStatement(sql)) {
   stmt.setString(1, user.getName());
   stmt.setString(2, user.getEmail());
   stmt.executeUpdate();
  }
 }
}
```

Exemple DTO en Java

```java
public class UserDTO {
    private int id;
    private String name;
    private String email;

    // Getters et Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

Où utilise-t-on DAO et DTO ?

DAO (Data Access Object)

- Utilisé dans la **couche d'accès aux données**.
- Abstraction de la source de données (BDD, API, fichiers, etc.).
- Exemple : Accès à une base de données avec JDBC, Hibernate, ou un ORM.

Où utilise-t-on DAO et DTO ? -suite-

DTO (Data Transfer Object)

- Utilisé pour **transférer des données** entre couches (ex : entre la couche service et la couche présentation).
- Évite d'exposer les objets métier ou les entités issues de la bdd directement.
- Exemple : retourner un objet User avec des informations non-sensibles (retirer le password, ...).
- Exemple : réponse d'une API REST (alléger les réponses), données envoyées à une vue.

Où placer la frontière de transaction ?

- Le code JDBC des transactions (`setAutoCommit`, `commit`, `rollback`) a été vu au chapitre 2
- Un DAO = une table/entité. Or un cas d'usage métier touche souvent plusieurs DAO

⇒ La transaction se démarre et se termine dans la **couche service**, et les DAO partagent la même connexion

```
public void virer(int src, int dst, double montant) throws SQLException {
  try (Connection c = dataSource.getConnection()) {
    c.setAutoCommit(false);
    try {
      compteDAO.debiter(c, src, montant);
      compteDAO.crediter(c, dst, montant);
      operationDAO.journaliser(c, src, dst, montant);
      c.commit();
    } catch (SQLException e) { c.rollback(); throw e; }
  }
}
```

Avec JPA ou Spring : même principe (`EntityTransaction`, puis `@Transactional` sur le service).

Gestion des exceptions

Ne pas exposer les exceptions SQL dans les couches supérieures. Utiliser des exceptions métiers ou des DTOs pour retourner des erreurs

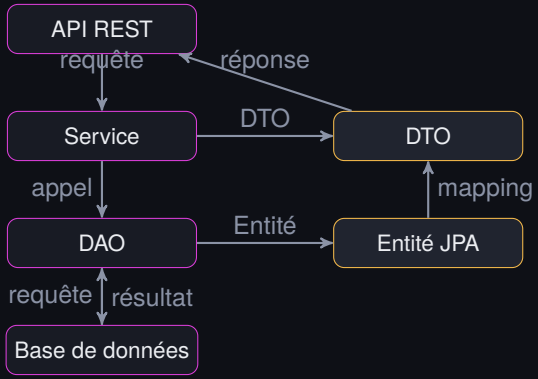
```
public UserDTO getUserById(int id) {
    try {
        // Logique SQL
    } catch (SQLException e) {
        throw new DataAccessException("Erreur lors de la recuperation de l'utilisateur", e)
     ;
    }
}
```

Structure typique d’une application



Les DTO circulent entre les couches, les DAO interagissent avec la bdd.

Exemple d’architecture avec DAO et DTO



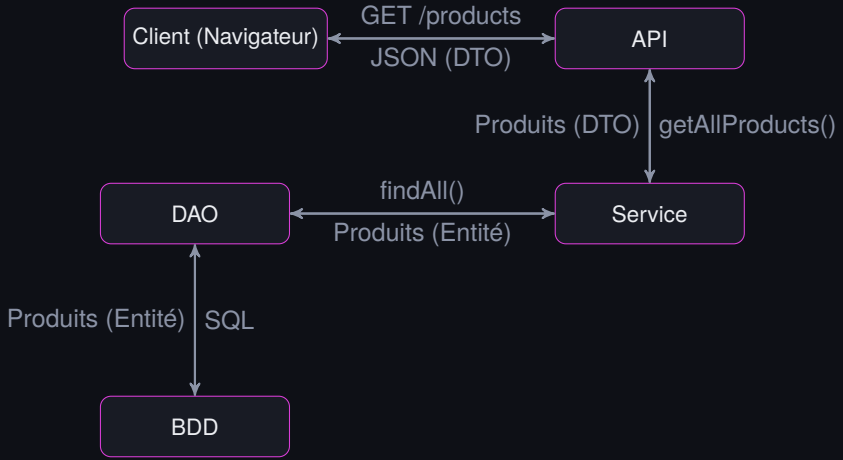
Les DTO sont utilisés pour les échanges entre l’API et le service, et entre le service et la vue.

Cas d’usage concret : Application Web

Scénario

- Un utilisateur demande la liste des produits via une API REST.
- Le **Service** appelle le **DAO** pour récupérer les données.
- Le **DAO** interroge la base de données et retourne une **Entité**.
- Le **Service** convertit l’Entité en **DTO** et renvoie la réponse.

Cas d’usage concret : Application Web



Avantages de cette structure

DAO

- Découplage de la logique métier et de l'accès aux données.
- Facilite les tests unitaires (mock des DAO).
- Permet de changer de source de données sans impacter le reste.

DTO

- Contrôle précis des données exposées.
- Réduction du trafic réseau (seulement les données nécessaires).
- Sécurité (masquage des données sensibles).

DAO + DTO = Architecture modulaire, maintenable et sécurisée.

Structure de dossiers pour une application Java

```
src/
|-- main/
|   |-- java/com/monapplication/
|   |   |-- config/
|   |   |-- controller/
|   |   |-- service/
|   |   |-- model/
|   |   |   |-- entity/
|   |   |   |-- dao/
|   |   |   |-- dto/
|   |   |   `-- exception/
|   |   |-- util/
|   |   `-- Application.java
|   `-- resources/
|       |-- application.properties
|       `-- db/
`-- test/
```

Explications

- `config/` : Configuration de l'application.
- `controller/` : Contrôleurs pour les requêtes HTTP.
- `service/` : Logique métier.
- `model/entity/` : Entités JPA.
- `model/dao/` : Accès aux données.
- `model/dto/` : Objets de transfert de données.
- `model/exception/` : Exceptions personnalisées.
- `util/` : Utilitaires et helpers.
- `resources/` : Fichiers de configuration et scripts SQL.
- `test/` : Tests unitaires et d'intégration.

Bonnes pratiques pour une architecture Java

Séparation des couches

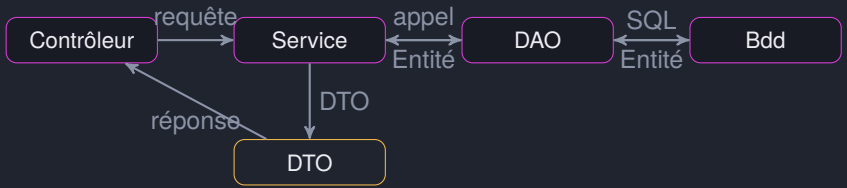
- **Contrôleurs** : Gèrent les requêtes HTTP (API REST ou MVC).
- **Services** : Contiennent la **logique métier** et utilisent les DAO.
- **DAO** : Encapsulent les **requêtes SQL/JPA** et l'accès aux données.
- **DTO** : Transfèrent les données entre couches **sans logique métier**.
- **Entités** : Représentent les **tables de la base de données** (JPA).

Bonnes pratiques pour une architecture Java -suite-

Règles à respecter

- Ne **jamais** exposer les entités JPA directement dans une API.
- Utiliser des **DTO** pour les réponses API.
- Gérer les **transactions** au niveau des services.
- **Injecter les dépendances** (ex : `@Autowired` avec Spring).
- **Ne pas** appeler un DAO directement depuis un contrôleur.

Exemple de flux



Conclusion

- DAO et DTO sont complémentaires pour une architecture propre.
- DAO gère l'accès aux données, DTO transporte les données.
- Utilisables dans de nombreux langages (Java, TypeScript, Python, etc.).

Questions pour vous, alors

1. Pourquoi utiliser un DAO plutôt que d'écrire des requêtes SQL directement dans le code métier ?
2. Comment un DTO peut-il améliorer les performances d'une application ?
3. Quels sont les risques de ne pas utiliser de transactions dans un DAO ?
4. Comment adapteriez-vous un DAO pour utiliser une base de données NoSQL (ex : MongoDB) ?

Questions ?



Chouki.Tibermacine@univ-ubs.fr